

Collections Framework: Week 4 Unit Testing

VERIFYING ARRAYLIST DYNAMIC RESIZING & MEMORY SAFETY WITH UNITY

In Week 4 of the **CCollections** semester project, we implement index-specific behaviors on top of our generic Collection hierarchy. Testing a generic `void*` implementation requires both logic verification (bounds checking and capacity scaling) and rigorous memory isolation validation.

DevOps Objective: The Unity testing suite below is fully integrated with your CMake environment. When combined with the `tests/run_tests.sh` script under Valgrind, any allocation leakage or indexing overflows will automatically halt the pipeline build.

1. Creating the Unity Test Suite Structure

To implement this, you will create two files inside your repository: a mock setup header containing type configurations and your main unit test wrapper file.

File Setup: `tests/test_arraylist.c`

This test file initializes your generic `List` interface, pushes elements beyond the initial 10-slot layout capacity, and ensures that dynamic resizing relocates heap addresses safely without leaking underlying contexts.

```

#include "unity.h"
#include "list.h"
#include <stdlib.h>

// Shared pointer allocations for testing
static List *test_list = NULL;

// Mock payload struct to verify multi-byte pointer handling
typedef struct {
    int id;
    char *name;
} MockItem;

// Free payload function matching our defined FreeFunc signature
void free_mock_item(void *item) {
    if (item) {
        free((MockItem*)item->name);
        free(item);
    }
}

// Executed automatically before each single test scenario runs
void setUp(void) {
    // External function from src/arraylist.c
    extern List* create_arraylist(void);
    test_list = create_arraylist();
    TEST_ASSERT_NOT_NULL(test_list);
}

// Executed automatically after each test finishes to purge dynamic memory leak risks
void tearDown(void) {
    if (test_list) {
        test_list->collection.clear(&(test_list->collection), free_mock_item);
        free(test_list->collection.collection_context);
        free(test_list);
        test_list = NULL;
    }
}

// TEST 1: Assure initial sizing constraints hold true
void test_arraylist_initial_state(void) {
    TEST_ASSERT_EQUAL_UINT(0, test_list->collection.size(&(test_list->collection)));
}

// TEST 2: Validate automatic sizing expansion and retention under capacity overflow
void test_arraylist_dynamic_resizing_boundary(void) {
    // Exceed initial threshold (ARRAYLIST_INITIAL_CAPACITY is 10)
    const int element_count = 25;

    for (int i = 0; i < element_count; i++) {
        MockItem *item = malloc(sizeof(MockItem));
        item->id = i;
        item->name = malloc(16);
        sprintf(item->name, "Item %d", i);

        bool success = test_list->collection.add(&(test_list->collection), item);
        TEST_ASSERT_TRUE(success);
    }

    // Check that the collection reports correct track size

```

```

    TEST_ASSERT_EQUAL_UINT(element_count, test_list->collection.size(&(test_list->collection)));

    // Verify values at random indices to confirm pointer array remains intact after resizing
    MockItem *seventh = (MockItem*)test_list->get(test_list, 7);
    TEST_ASSERT_NOT_NULL(seventh);
    TEST_ASSERT_EQUAL_INT(7, seventh->id);
    TEST_ASSERT_EQUAL_STRING("Item 7", seventh->name);

    MockItem *last = (MockItem*)test_list->get(test_list, 24);
    TEST_ASSERT_NOT_NULL(last);
    TEST_ASSERT_EQUAL_INT(24, last->id);
    TEST_ASSERT_EQUAL_STRING("Item 24", last->name);
}

// TEST 3: Validate out-of-bounds error-handling bounds safety
void test_arraylist_out_of_bounds_protection(void) {
    // ArrayList is empty right now, index 0 should securely return NULL
    TEST_ASSERT_NULL(test_list->get(test_list, 0));
    TEST_ASSERT_NULL(test_list->get(test_list, 999));
}

int main(void) {
    UNITY_BEGIN();
    RUN_TEST(test_arraylist_initial_state);
    RUN_TEST(test_arraylist_dynamic_resizing_boundary);
    RUN_TEST(test_arraylist_out_of_bounds_protection);
    return UNITY_END();
}

```

2. How the DevOps Pipeline Executes this Testing Sequence

When you push your commits to GitHub, the **CMake** runner resolves dependency linkages, compiles your test executable `run_tests`, and runs your checks inside the **Valgrind** engine:

```
valgrind --leak-check=full --show-leak-kinds=all --errors-returning-error=all --error-exitcode=1 ./run_tests
```

Because our `tearDown()` sequence calls `test_list->collection.clear()` with a valid custom `free_mock_item()` destruct callback, Valgrind guarantees that every nested dynamically allocated structure (such as the strings inside `name`) is completely reclaimed. This gives your CCollections library safety assurances equivalent to managed languages.